ASSIGNMENT 2 / CONVOLUTION

Cats vs Dogs
Convolutional Neural Networks

How training sample size changes the performance of a CNN trained from scratch versus a pretrained MobileNetV2

ACADEMIC RESEARCH REPORT

Based on the final executed Kaggle notebook and its recorded validation, test, holdout, and data-audit results.

8	96.4%	28/28
CNN configurations	Pretrained 2K test accuracy	Exact-content audit passes

Primary research question
When labeled training data are limited, should an image classifier be learned from scratch or built on pretrained visual features?

Scope Six assignment-required experiments, two larger scratch CNNs, dropout and seed-stability studies, two additional holdouts, and a post-experiment image-integrity audit.

Prepared by: Aamir Sohaib | Report date: October 24th, 2026

Evidence repository: github.com/roceye1/811424116_AdvancedMachineLearning

The final executed notebook is the canonical source for measured results. Supplementary training sizes are explicitly distinguished from the professor-defined 2,000-image pool.

Executive summary

I compared two approaches to classifying cat and dog images: a CNN trained from scratch and a classifier using frozen ImageNet-pretrained MobileNetV2 features. The core study used balanced, nested training sets of 1,000, 1,500 and 2,000 labeled images. The same 500-image validation and 500-image test sets were used throughout.

On the fixed test set, MobileNetV2 achieved 96.2%–96.6% accuracy across the three core training sizes. The scratch CNN achieved 73.2%–77.0%. With 2,000 training images, their scores were 96.4% and 77.0%, a gap of 19.4 percentage points. Because these models also differ in architecture, the gap cannot be attributed solely to pretraining.

When I increased scratch-CNN training to 12,000 and 18,000 images, its original-test accuracy rose to 92.0% and 93.2%. The 18,000-image model was still 3.2 points below MobileNetV2 trained on 2,000 task-specific images. Separate dropout and seed experiments showed small, variable effects. In a model-specific SHA-256 audit, none of the 28 training–evaluation comparisons had byte-identical image overlap.

Results at a glance

Architecture	Training images	Validation	Original test
Scratch CNN	1,000	76.8%	74.2%
Scratch CNN	1,500	77.0%	73.2%
Scratch CNN	2,000	79.2%	77.0%
Pretrained MobileNetV2	1,000	97.4%	96.2%
Pretrained MobileNetV2	1,500	97.4%	96.6%
Pretrained MobileNetV2	2,000	97.6%	96.4%

Within the required 2,000-image pool, validation accuracy for the scratch CNN rose as training size increased. MobileNetV2 performed well even at 1,000 images and changed little across the three sizes. The 1,500-image pretrained model scored one test image above the 2,000-image model. I kept model selection based on validation rather than choosing retrospectively by test score.

How to read this report

Sections 1–4 address the assignment questions. Sections 5–7 cover the additional scaling, dropout, holdout and image-quality analyses. Section 8 explains the limitations, followed by the conclusion, references and detailed results.

1. Research objective and assignment alignment

The assignment asked me to use convnets for image classification and compare training from scratch with a pretrained network as training size changes. Questions 1–3 vary the scratch CNN training sample; Question 4 applies comparable sample sizes to a pretrained model. I also examined augmentation and regularisation as ways to limit overfitting.

Study questions

- Q1: What performance is achieved by a scratch CNN with 1,000 training, 500 validation and 500 test images?
- Q2: What happens when the scratch training sample is increased while validation and test images remain fixed?
- Q3: Which training size among the required configurations improves on the previous scratch results?
- Q4: How do equivalent sample-size experiments perform with a pretrained convolutional network?

The 1K, 1.5K and 2K runs form the required analysis under the clarified 2K training pool. The 12K and 18K scratch models, dropout study and error analysis are separate extensions, not replacements for those runs.

2. Dataset, sample design and preprocessing

2.1 Dataset and class labels

I used the labeled train.zip archive from Kaggle Dogs vs. Cats: 25,000 JPEG files, evenly divided between cats and dogs. Labels came from the filename prefixes. I created evaluation sets from this labeled archive rather than from the competition's unlabeled test archive. Cat was coded 0 and Dog 1. [1]

2.2 Fixed and supplementary partitions

Partition	Count	Purpose
Core training pool	2,000 (1,000 per class)	Nested 1K, 1.5K and 2K training samples
Original validation	500 (250 per class)	Checkpoint selection across core and extended models
Original test	500 (250 per class)	Final eight-model comparison
Extended training additions	16,000	Construct 12K/18K scratch selections
Extended validation	2,000	Additional check for large scratch models
Holdout 1	2,000	Cross-model generalization/error study
Holdout 2	2,000	Distinct dropout-specific evaluation

The nested samples contained 500 cats and 500 dogs at 1K, 750 of each at 1.5K, and 1,000 of each at 2K. The 12K and 18K scratch samples contained 6,000 and 9,000 per class. Frozen filename manifests were checked for class balance and disjoint filenames. Holdout 1 and Holdout 2 are different image cohorts; their results are not interchangeable.

2.3 Image pipeline

I decoded images as three-channel RGB and resized them to 150 × 150 pixels. Training batches were shuffled, but evaluation images were not augmented. The scratch CNN rescaled pixels by 1/255; MobileNetV2 used preprocess_input. Horizontal flips, rotations (0.06 factor) and zoom (0.10 factor) were applied only in training. Each architecture needs its own preprocessing for a fair comparison.

3. Model architectures and evaluation protocol

3.1 CNN trained from scratch

The scratch CNN used four convolution/max-pooling blocks with 32, 64, 128 and 128 filters, followed by Flatten, a 128-unit ReLU layer, dropout of 0.50 in the core runs, and one sigmoid output. Each model learned its weights from random initialisation. Training used binary cross-entropy and Adam at a 0.0003 learning rate.

3.2 Pretrained MobileNetV2

For the other model, I froze an ImageNet-pretrained MobileNetV2 backbone and added global average pooling, 0.50 dropout and a sigmoid classifier. MobileNetV2 uses inverted residual and linear bottleneck modules (Sandler et al., 2018). Keras loaded the default 224-resolution pretrained weight variant, but the network processed 150 × 150 inputs with its classification head excluded.

3.3 Training and model selection

The principal models trained for 25 epochs with batches of 32 and shared the original validation set. ModelCheckpoint saved the epoch with lowest validation loss, and I tested that saved checkpoint. Reported measures are accuracy, binary cross-entropy loss, precision, recall and F1. Dog is the positive class for precision, recall and F1; F1 is the harmonic mean of precision and recall.

The main seed was 42. I recorded manifest fingerprints and checked checkpoint hashes; later repeats used seeds 17, 42 and 123. Fixed seeds help replication but cannot remove every source of GPU nondeterminism. Since the two models differ in both architecture and pretrained weights, this comparison measures their practical performance, not the isolated effect of pretraining.

4. Required assignment results: Questions 1–4

4.1 Q1–Q3: Scratch CNNs at 1K, 1.5K and 2K

Question	Training	Best epoch	Validation acc.	Test acc.	Test loss	Test F1
Q1	1,000	25	76.8%	74.2%	0.5727	0.7495
Q2	1,500	24	77.0%	73.2%	0.5690	0.7265
Q3	2,000	25	79.2%	77.0%	0.4865	0.7619

Q1 reached 74.2% test accuracy using 1,000 labeled training images. Increasing the sample to 1,500 (Q2) slightly improved validation accuracy to 77.0%, but observed test accuracy declined to 73.2%. The Q3 model at 2,000 images achieved the highest scratch accuracy among the required sample sizes on both validation (79.2%) and

test (77.0%). Thus, the measured optimum within the specified 1K–2K search was 2,000 images; no claim is made about a globally optimal dataset size.

Adding images did not raise every test score. Q2 improved validation accuracy, yet its test accuracy fell slightly. A 500-image test set, differences in sample composition and training randomness can all affect a small change like this. The overall pattern for this scratch architecture favors more data, but it is not monotonic.

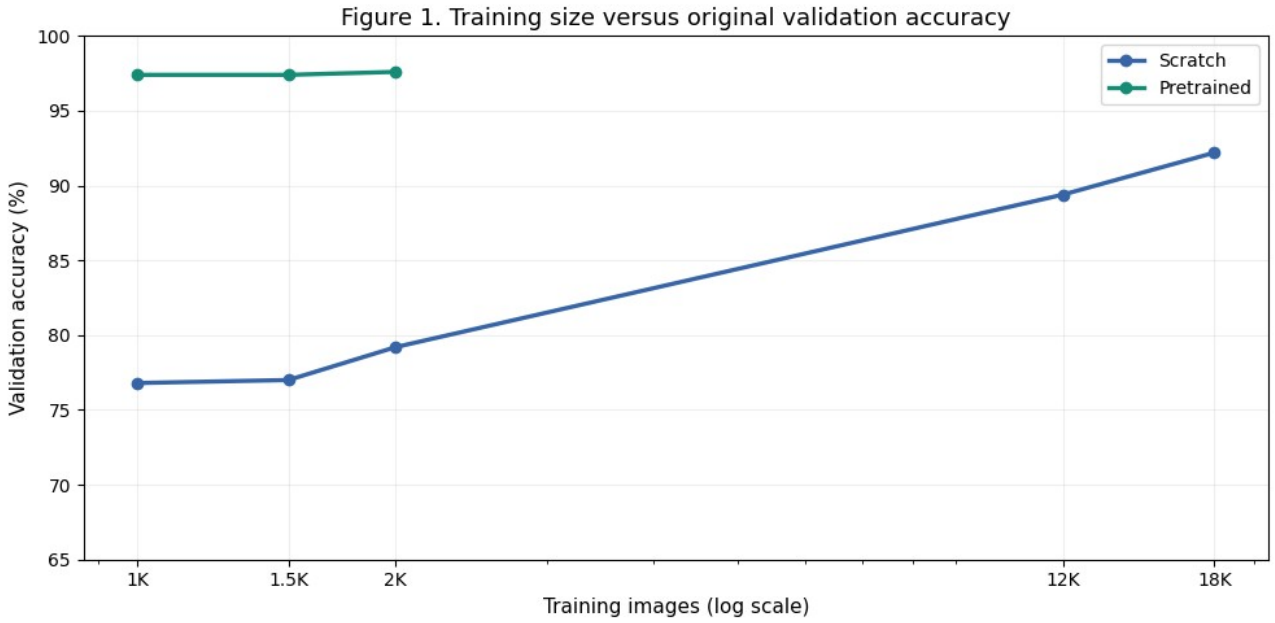


Figure 1. Validation accuracy versus training-set size. Scratch CNN improves with more data, while pretrained accuracy is nearly flat at the core sizes.

4.2 Q4: Pretrained MobileNetV2 at corresponding sizes

Training	Best epoch	Validation acc.	Test acc.	Test loss	Test F1
1,000	24	97.4%	96.2%	0.0857	0.9622
1,500	20	97.4%	96.6%	0.0787	0.9663
2,000	25	97.6%	96.4%	0.0699	0.9641

MobileNetV2 reached 97.4%–97.6% validation accuracy and 96.2%–96.6% on the original test across the core sizes. The 2K model had the strongest validation accuracy and lowest validation loss. The 1.5K model led on the test by 0.2 percentage points, just one image; I did not revise the validation-based selection on that basis.

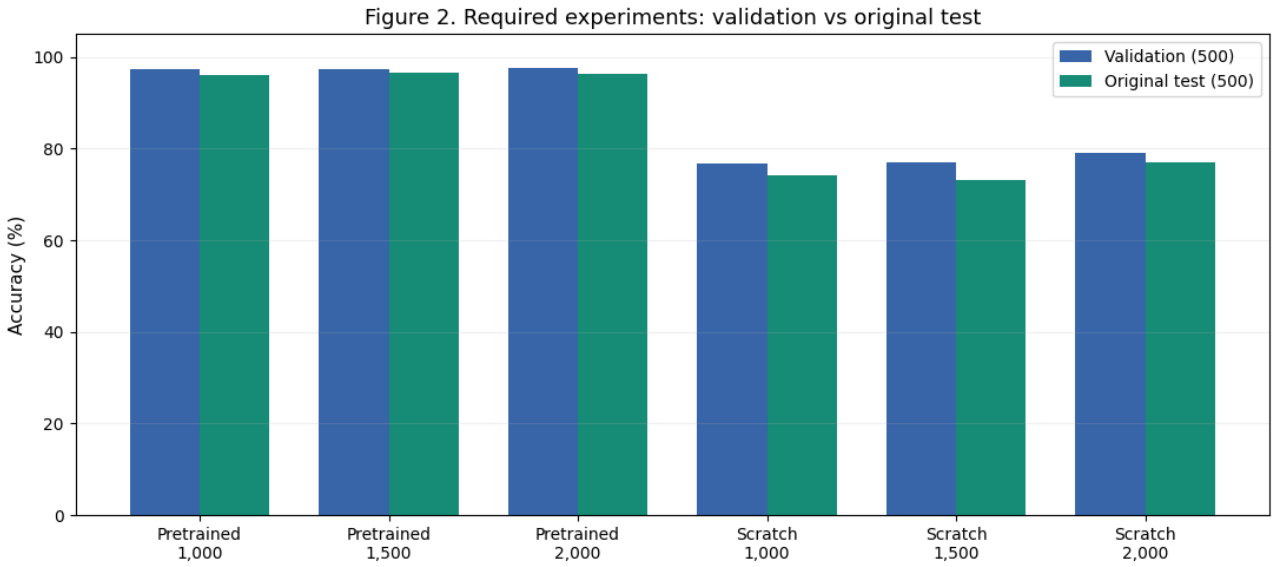


Figure 2. Six professor-required runs on the same validation and original test cohorts. The performance separation between model families is substantial.

4.3 Direct response to the assignment question

At matching training sizes, MobileNetV2 exceeded the scratch CNN on test accuracy by 22.0 points at 1K, 23.4 at 1.5K and 19.4 at 2K. In this cats-versus-dogs study, pretrained features were more useful when labeled task data were limited. The scratch CNN improved more substantially only when I trained it with far more images.

5. Supplementary extension: 12K and 18K scratch CNNs

I trained two additional scratch CNNs with 12,000 and 18,000 images to see whether a much larger dataset narrowed the gap. These runs extend beyond the clarified 2,000-image assignment pool. They retain the core architecture and use the original 500-image validation and test sets.

Scratch training	Original validation	Extra validation	Original test	Test F1	Train time
12,000	89.4%	91.35%	92.0%	0.9234	4.57 min
18,000	92.2%	93.70%	93.2%	0.9331	6.77 min

Compared with scratch 2K, scratch 18K improved original-test accuracy by 16.2 points, from 77.0% to 93.2%. MobileNetV2 trained with 2K still led by 3.2 points on that same test set (96.4%). It used one-ninth as many task-specific training images, although architecture as well as pretraining differs.

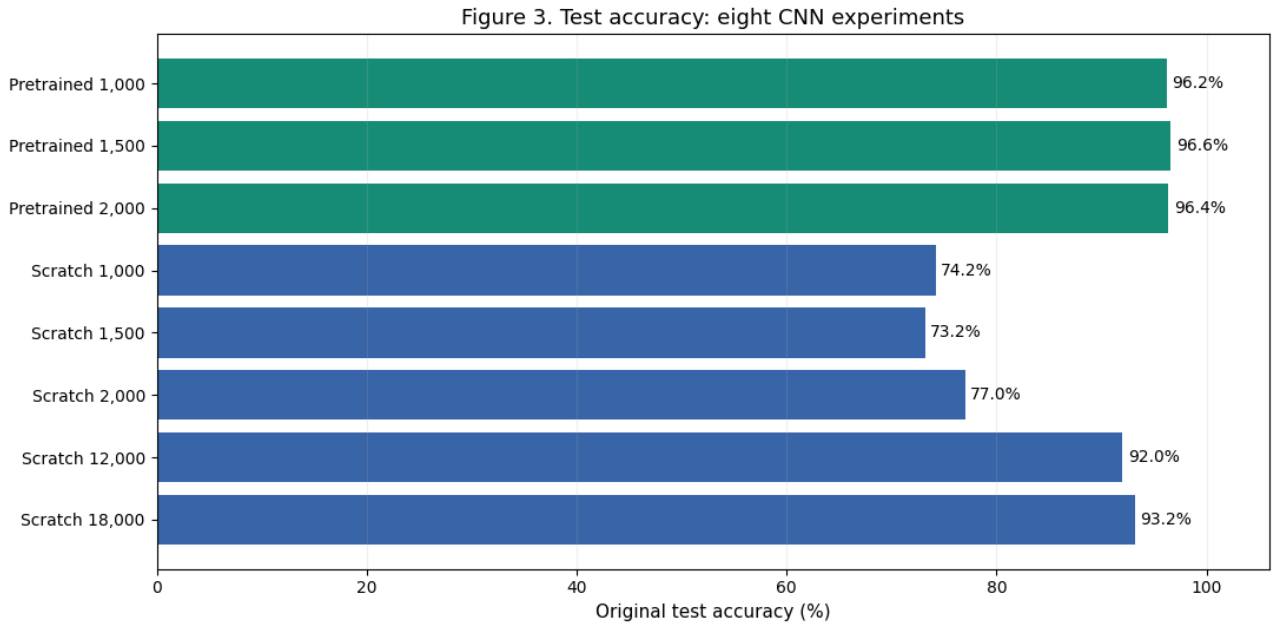


Figure 3. Original-test accuracy for all eight trained configurations. The 12K/18K runs are supplementary and should not be mistaken for required Q3 sample sizes.

5.1 Learning dynamics

The saved learning curves show MobileNetV2 reaching high validation accuracy early and maintaining low validation loss. Scratch 2K improved gradually but stayed below 80% validation accuracy. Scratch 18K had a much stronger trajectory, with some epoch-to-epoch variation. These curves describe the recorded runs, not confidence intervals or guaranteed behavior under other seeds.

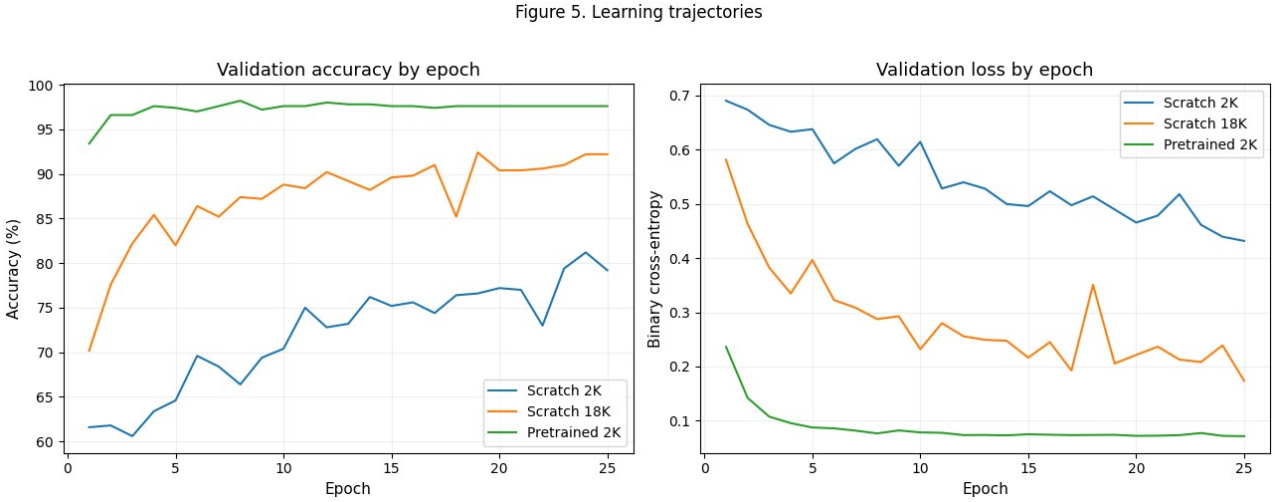


Figure 4. Saved per-epoch validation accuracy and loss for scratch 2K, scratch 18K and pretrained 2K. Curves use the same 25-epoch budget.

6. Supplementary robustness: holdouts, dropout and random seeds

6.1 Generalization on a second 2,000-image cohort

Model	Original test (500)	Holdout 1 (2,000)	Holdout 1 F1	Holdout errors
Scratch 2K	77.0%	77.95%	0.7733	441
Scratch 12K	92.0%	91.40%	0.9167	172
Scratch 18K	93.2%	93.10%	0.9313	138
Pretrained 2K	96.4%	97.00%	0.9699	60

On the separate 2,000-image Holdout 1, the ranking stayed the same: pretrained 2K scored 97.0%, followed by scratch 18K at 93.1%, scratch 12K at 91.4% and scratch 2K at 77.95%. This offers another check on the result, but label noise and repeated inspection during exploration still limit what we can conclude.

Figure 4. Generalization across fixed evaluation cohorts

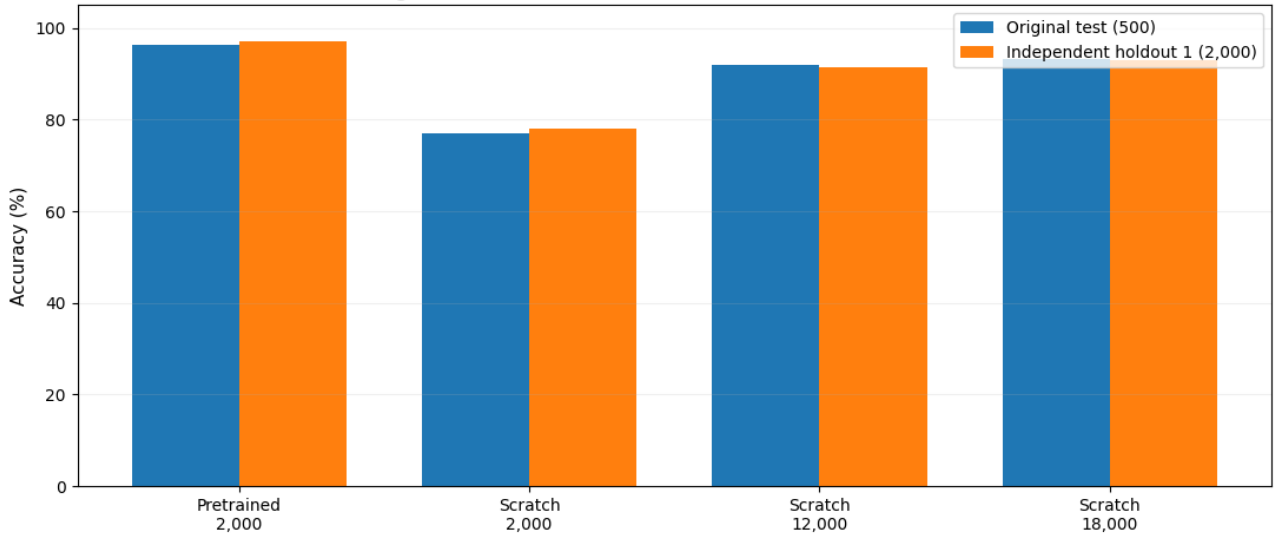


Figure 5. Comparison across the original 500-image test and the distinct 2,000-image Holdout 1 for the four models evaluated on both cohorts.

6.2 Dropout-rate sensitivity

I trained three scratch CNNs on the same 2K pool with dropout set to 0.00, 0.25 or 0.50. Dropout temporarily switches off some units during training to reduce dependence on particular feature combinations (Srivastava et al., 2014). I evaluated them on Holdout 2, a different 2,000-image cohort from Holdout 1.

Dropout	Training acc.	Validation acc.	Train–val gap	Holdout 2 acc.	Holdout 2 errors
0.00	88.95%	79.6%	9.35 pts	78.95%	421
0.25	87.00%	79.8%	7.20 pts	78.80%	424
0.50	86.75%	80.2%	6.55 pts	79.45%	411

In this single-run comparison, dropout 0.50 gave the smallest training–validation accuracy gap and the best Holdout 2 result. Its edge over dropout 0.00 was 0.50 percentage points, or ten images out of 2,000. The narrower gap is consistent with stronger regularisation; it does not prove 0.50 is the best setting generally.

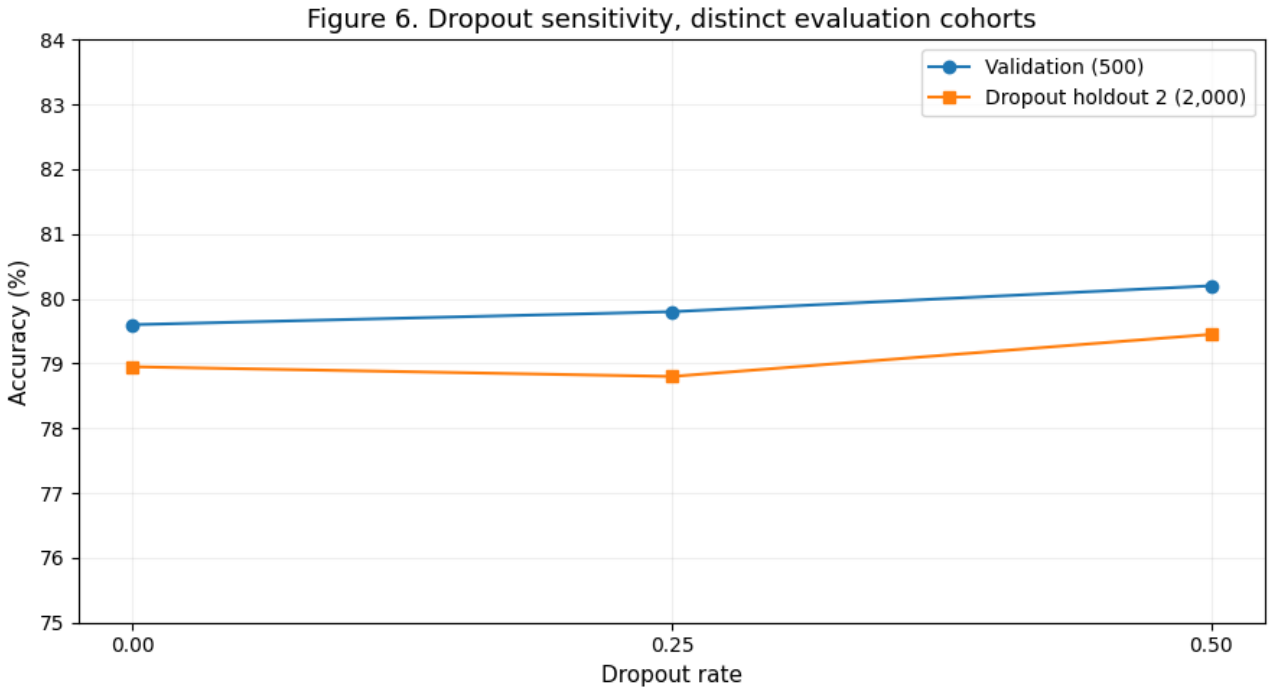


Figure 6. Single-seed dropout study comparing original validation (500) with the distinct dropout-specific Holdout 2 (2,000).

6.3 Three-seed stability study

I repeated the scratch experiment at dropout 0.00, 0.25 and 0.50 with seeds 17, 42 and 123: nine additional runs in total. All used the original validation set. Mean validation accuracies differed by less than 0.34 percentage points, while variation across runs was as large as, or larger than, the difference between settings.

Dropout	Mean validation	SD (points)	Mean val. loss	Mean gap
0.00	79.40%	0.72	0.4301	9.75 pts
0.25	79.33%	0.46	0.4324	8.52 pts
0.50	79.67%	0.83	0.4303	7.30 pts

Dropout 0.50 had the highest mean validation accuracy and smallest mean training–validation gap over these seeds, but also the largest accuracy standard deviation. With only three seeds per setting, I cannot conclude that it consistently beats the others. The measured performances were similar.

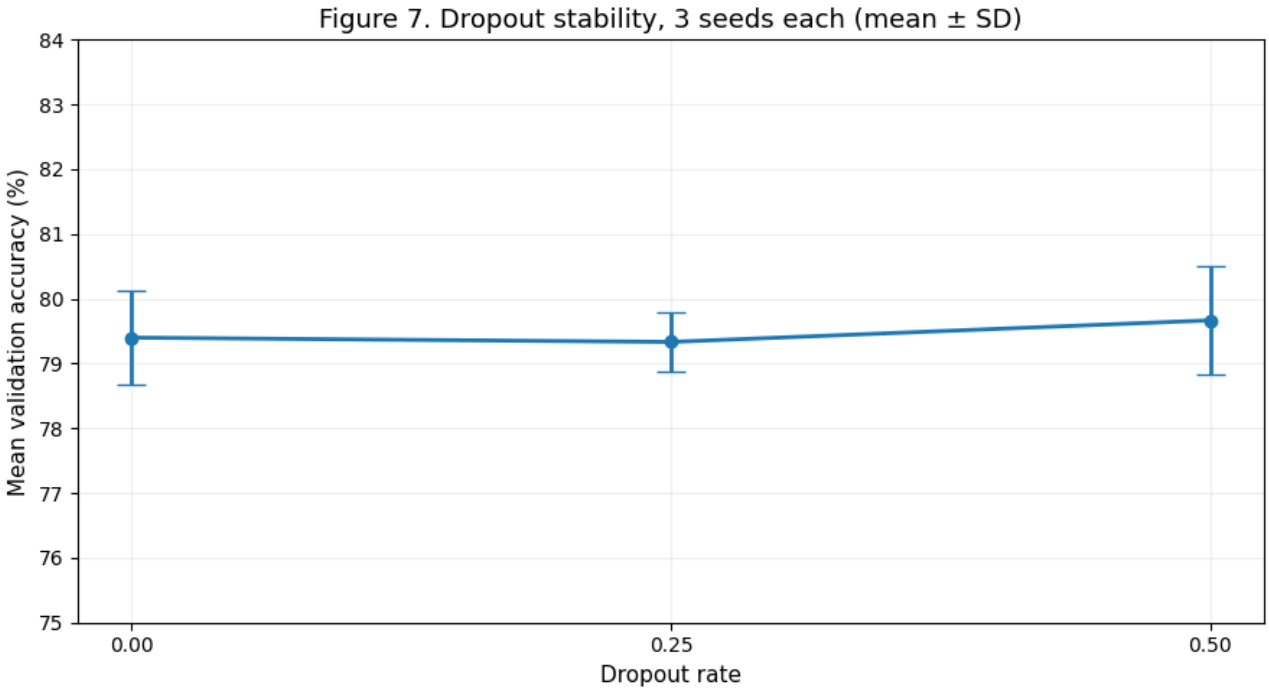


Figure 7. Mean validation accuracy with ±1 sample standard deviation across three predetermined seeds per dropout rate.

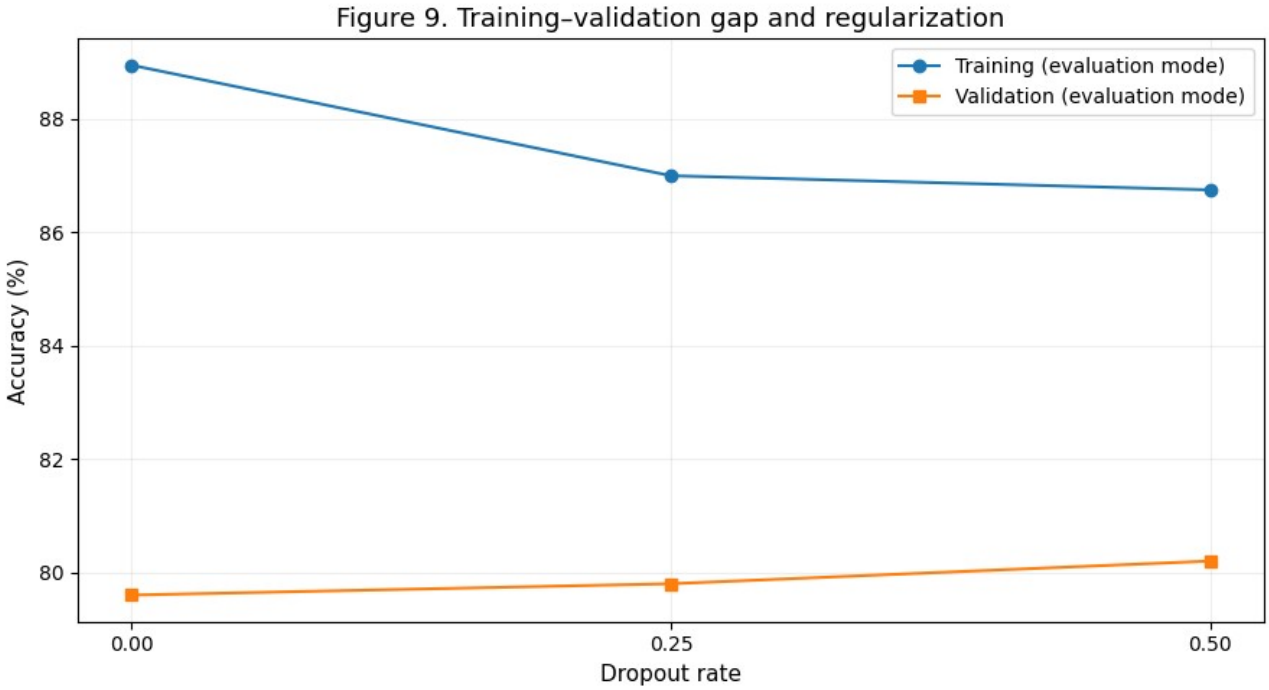


Figure 8. Single-seed training-versus-validation accuracy by dropout rate, showing reduced generalization gaps under stronger dropout.

7. Misclassification patterns and dataset quality

7.1 Which models made fewer mistakes?

On Holdout 1, scratch 2K misclassified 441 images, scratch 12K missed 172, scratch 18K missed 138 and pretrained 2K missed 60. The scratch models made fewer errors as training size grew, but the pretrained model still made the fewest on this shared cohort.



Figure 9. Incorrect classifications on the same balanced 2,000-image Holdout 1; all bars share the same denominator.

7.2 Error overlap and difficult images

Of the Holdout 1 images, 23 were misclassified by both scratch 18K and pretrained 2K. MobileNetV2 was correct on 115 images scratch 18K missed; scratch 18K was correct on 37 MobileNetV2 missed. All four selected models missed 14 images. The errors overlap partly, but these counts alone do not show why the images were difficult.

7.3 Suspicious source labels

A manual review flagged train/cat.4085.jpg as a likely source-label error: its filename says Cat, while the photo appears to show a dog. The local file matched the original Kaggle ZIP byte for byte, and all four models predicted Dog. This raises a data-quality question rather than proving a model error. An apparent pet-product logo appeared in an exploratory gallery, but its original filename was not established in the final audit.

I retained the original labels and reported metrics. Changing headline scores after reviewing test errors would introduce post-hoc bias. A corrected-label benchmark would need a predefined, independent label audit.

8. Data-integrity audit and methodological limitations

8.1 Exact-content duplicate-image audit

Filename checks first showed disjoint partitions. The later SHA-256 audit found five groups of byte-identical images under eleven filenames, something filename checks cannot detect. Extended validation had 1,999 distinct image contents among 2,000 rows. The 12K and 18K scratch training sets contained three and four repeated-content occurrences, respectively.

I then compared each fitted model's training-image hashes with only the evaluation cohorts used to score that model. All 28 model–evaluation comparisons had zero byte-identical overlap. A duplicate shared by extended training and Holdout 2 therefore did not create leakage in these evaluations: the dropout models assessed on Holdout 2 trained on the core 2K pool, while the larger scratch models were not assessed there.

Audit result	Verified value
Model/evaluation comparisons checked	28
Exact train–evaluation hash overlaps	0
Duplicate-content groups across full archive	5
Filenames in duplicate groups	11
Repeated training occurrences in scratch 12K / 18K	3 / 4
Repeated observation in extended validation	1

That finding is limited to exact byte matches. SHA-256 will not detect the same photograph if it has been resized, cropped or re-encoded. Repeated content within training sets also makes the number of unique images slightly smaller than the nominal sample count.

8.2 Limits on generalization and interpretation

- Architecture confounding: scratch CNN and MobileNetV2 differ both in architectural design and whether external ImageNet features were learned beforehand; no controlled architecture-matched pretraining ablation was performed.
- Restricted core search: the Q3 conclusion identifies the best among 1K, 1.5K and 2K, not an absolute optimal training size. The supplementary 12K/18K results illustrate further scaling but belong outside the assignment-defined 2K pool.
- Finite evaluation: the original test has 500 images, so one additional error changes measured accuracy by 0.2 percentage points. The 96.6% versus 96.4% pretrained distinction is one image and should not be overinterpreted.
- Comparability and compute: the experiments used 25 epochs per model, meaning larger datasets also yielded more optimizer updates; image-count effects were not completely disentangled from training computation.
- Repeated benchmark inspection: additional exploratory comparisons followed earlier evaluations. Reported original tests are descriptive and must not be reused for subsequent hyperparameter tuning.
- Data noise and exact-hash scope: at least one likely mislabel was flagged manually, repeated image contents exist, and perceptual near-duplicates were not comprehensively tested.

9. Conclusion and recommendations

With limited task-specific labels, pretrained MobileNetV2 classified these cats and dogs more accurately than the scratch CNN used here. At 2,000 training images, the two models scored 96.4% and 77.0%, respectively, on the same original 500-image test set. Within the required sample sizes, 2K gave the strongest scratch validation and test results; pretrained accuracy was already around 96% at 1K.

More training data helped the scratch model: its test accuracy rose to 93.2% at 18K, though it remained below pretrained 2K. The nine-run dropout study found relatively small differences between settings and did not justify picking a rate from one run alone. The additional holdout and duplicate-content checks also show why evaluation design and image quality matter.

For a similar small-data task, I would begin with a suitable pretrained network and its required preprocessing, fix the validation and test protocol, and compare it against a documented scratch baseline. I would also check data quality independently. If a scratch model is required, the amount and quality of labeled data may make a substantial difference. These are conclusions from the models and images studied here, not a universal ranking of transfer-learning methods.

References

[1] Kaggle. (n.d.). Dogs vs. Cats [competition dataset]. https://www.kaggle.com/competitions/dogs-vs-cats/data

[2] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 4510–4520. https://openaccess.thecvf.com/content_cvpr_2018/html/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.html

[3] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(56), 1929–1958. https://jmlr.org/papers/v15/srivastava14a.html

[4] Course assignment brief. Assignment 2: Convolution (two-page PDF).

[5] Assignment 2 — CNN Cats vs Dogs. (2026). Final executed Kaggle notebook and archived result tables. https://github.com/roceye1/811424116_AdvancedMachineLearning

Appendix A. Complete eight-model original-test metrics

Model	N	Val.	Test	Loss	Prec.	Recall	F1	Errors
Pretrained	1,000	97.40%	96.20%	0.0857	0.9565	0.9680	0.9622	19
Pretrained	1,500	97.40%	96.60%	0.0787	0.9569	0.9760	0.9663	17
Pretrained	2,000	97.60%	96.40%	0.0699	0.9603	0.9680	0.9641	18
Scratch	1,000	76.80%	74.20%	0.5727	0.7283	0.7720	0.7495	129
Scratch	1,500	77.00%	73.20%	0.5690	0.7417	0.7120	0.7265	134
Scratch	2,000	79.20%	77.00%	0.4865	0.7897	0.7360	0.7619	115
Scratch	12,000	89.40%	92.00%	0.1990	0.8860	0.9640	0.9234	40
Scratch	18,000	92.20%	93.20%	0.1726	0.9186	0.9480	0.9331	34

All eight models used the same original 500-image test set. The 12K and 18K configurations are supplementary. Precision, recall and F1 treat Dog as the positive class.

Appendix B. Nine-run dropout seed results

Dropout	Seed	Best epoch	Train acc.	Validation acc.	Val. loss	Gap
0.00	17	25	88.80%	78.60%	0.4450	10.20 pts
0.00	42	25	88.60%	79.60%	0.4210	9.00 pts
0.00	123	25	90.05%	80.00%	0.4250	10.05 pts
0.25	17	25	88.05%	79.60%	0.4300	8.45 pts
0.25	42	24	87.35%	78.80%	0.4360	8.55 pts
0.25	123	24	88.15%	79.60%	0.4310	8.55 pts
0.50	17	25	87.15%	79.00%	0.4210	8.15 pts
0.50	42	25	86.90%	80.60%	0.4320	6.30 pts
0.50	123	25	86.85%	79.40%	0.4380	7.45 pts

The three-seed repeat summary in Section 6.3 uses these nine runs; standard deviations are sample SDs.

Appendix C. Training-time observation and reproducibility

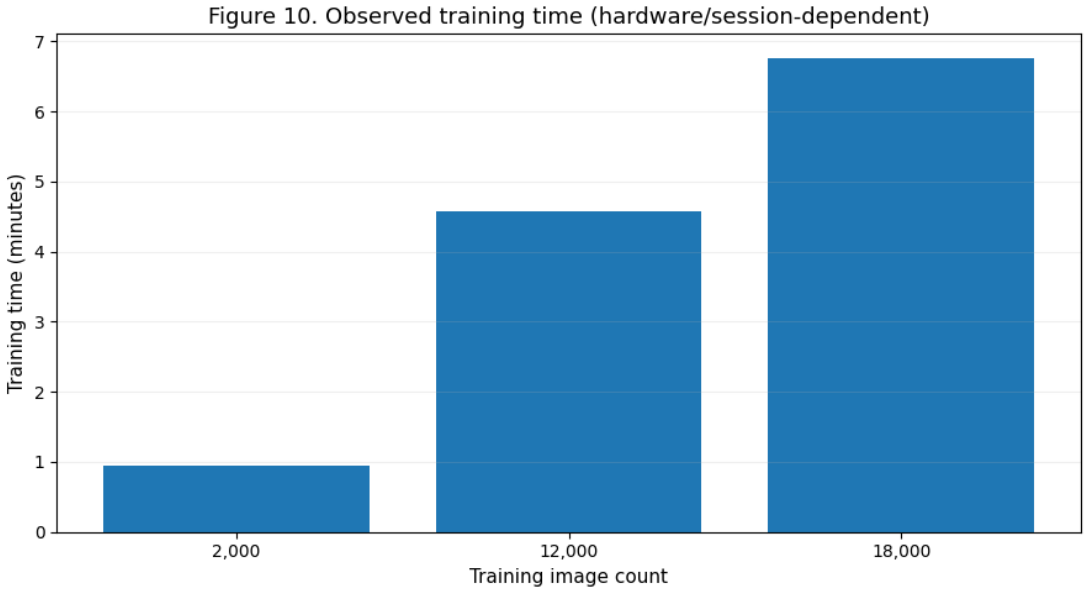


Figure 10. Observed training time in the recorded Kaggle session, shown descriptively. Time comparisons depend on hardware, caching and workload and are not an architecture-independent efficiency benchmark.

Source of record and submission artifacts

The notebook (notebooks/Assignment_2_CNN_Final_Executed.ipynb) records the code and outputs. Metrics are in results/*.csv, plots in figures/*.png, and hash-audit files in audit/*.csv and docs/DATA_INTEGRITY.md. The archive/ folder separately retains earlier exploratory work.

Execution environment: Kaggle; Python 3.13.15; TensorFlow 2.20.0; Pandas 2.3.3; 150 × 150 RGB; batch size 32; 25 epochs; Adam 0.0003; seed 42. Two T4 GPUs were visible, but distributed training was not established. Save the competition ZIP and full .keras checkpoints with the Kaggle outputs; they are not included in GitHub.

Repository clone URL: https://github.com/roceye1/811424116_AdvancedMachineLearning.git